KoTH Platform — Technical Documentation

Version: 2.0 (Generalized Architecture)
Project: github.com/Izhaan-Raza/KOTH_orchestrator
Stack: Python 3.11+, FastAPI, SQLite, Docker SDK, Paramiko, PyYAML

Table of Contents

1. Architecture Overview
2. Machine Registry
3. Machine Definition Spec (koth-machine.yaml)
4. Upload System
5. Deployment Engine
6. Port Manager
7. Adaptive Referee
8. Database Layer
9. Web UI
10. API Reference
11. Known Issues and Pitfalls
12. Security Considerations
13. Running in Production
14. Contributing a Machine

---

1. Architecture Overview

1.1 What Changed from v1

The v1 architecture hardcoded 24 machines across Series H1–H8. The referee knew exactly

which ports to poll and which SSH targets to reach because everything was written into `.env` at deploy time.

v2 inverts this. An organizer uploads any container they want. The platform stores it in a registry, deploys it to a node, and the referee adapts its poll list by reading from the registry at runtime instead of from a static config.

```
┌──────────────────────────────────────────────────────────────┐
│                     platform-server (:8000)                  │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌────────────────────┐  │
│  │  Registry    │  │  Deployment  │  │  Upload Handler    │  │
│  │  API         │  │  Engine      │  │  (YAML/Dockerfile/ │  │
│  │  (CRUD)      │  │  (Docker SDK)│  │   Compose/Web UI)  │  │
│  └──────┬───────┘  └──────┬───────┘  └────────────────────┘  │
│         │                 │                                  │
│  ┌──────▼─────────────────▼──────────────────────────┐       │
│  │              SQLite Registry DB                   │       │
│  │  machines | deployments | ports | teams | scores  │       │
│  └───────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────┘
                          │
              ┌───────────▼──────────┐
              │    referee-server    │
              │  polls /api/machines/ │
              │  active every N sec  │
              │  hot-reloads on change │
              └──────────┬───────────┘
                         │ SSH (Paramiko)
          ┌──────────────┼──────────────┐
          ▼              ▼              ▼
     [Node 1]       [Node 2]       [Node 3]
  any containers  any containers  any containers
  on dynamic ports on dynamic ports on dynamic ports
```

1.2 Component Responsibilities

Component	Responsibility
`platform-server/api/machines.py`	CRUD for machine registry
`platform-server/api/upload.py`	Accept and validate uploads
`platform-server/api/deploy.py`	Trigger deploy/undeploy on a node

`platform-server/engine/registry.py`	SQLite read/write for machine state
`platform-server/engine/deployer.py`	Docker SDK wrapper; SSH to remote node
`platform-server/engine/port_manager.py`	Track and assign ports across nodes
`referee-server/poller.py`	SSH-poll `king.txt` on active machines
`referee-server/scorer.py`	Award points; update leaderboard
`web-ui/`	Organizer-facing HTML dashboard

1.3 Data Flow for a New Machine

```
1. Organizer uploads koth-machine.yaml + Docker image (or Dockerfile or compose)
2. Upload handler validates YAML against schema; rejects unknown keys
3. Registry stores machine record with status = "registered"
4. Organizer clicks "Deploy to Node 2" in the web UI
5. deployer.py SSHes to node2; pulls image; runs container
6. port_manager.py records: machine_id → node2:10085
7. Registry updates status = "active"
8. referee poller's next iteration calls GET /api/machines/active
9. Poller adds node2:10085 to its SSH poll list
10. Teams attack; write /root/king.txt; poller reads it; scorer awards points
```

---

2. Machine Registry

2.1 What It Is

The registry is a SQLite database that is the single source of truth for every machine on the platform. The referee reads from it. The deployer writes to it. The API exposes it.

2.2 Schema

```
-- machines: one row per uploaded machine definition
CREATE TABLE machines (
    id          TEXT PRIMARY KEY,       -- UUID4, e.g. "a3f2c1..."
    name        TEXT NOT NULL,
    description TEXT,
    difficulty  TEXT CHECK(difficulty IN ('easy','medium','hard','insane')),
    points_per_tick INTEGER NOT NULL DEFAULT 10,
    king_file   TEXT NOT NULL DEFAULT '/root/king.txt',
```

```
    image_ref   TEXT,                   -- "myorg/vuln-nginx:latest" or NULL
    dockerfile  TEXT,                   -- path to Dockerfile or NULL
    compose_file TEXT,                  -- path to docker-compose.yml or NULL
    tags        TEXT,                   -- JSON array e.g. '["web","privesc"]'
    hints       TEXT,                   -- JSON array of hint strings
    status      TEXT NOT NULL DEFAULT 'registered',
    -- status values: registered | deploying | active | stopped | error
    uploaded_at TEXT NOT NULL,          -- ISO 8601
    uploaded_by TEXT                    -- organizer identifier
);

-- deployments: one row per running instance of a machine
CREATE TABLE deployments (
    id           TEXT PRIMARY KEY,      -- UUID4
    machine_id   TEXT NOT NULL REFERENCES machines(id),
    node_host    TEXT NOT NULL,         -- "192.168.1.10"
    node_ssh_user TEXT NOT NULL,        -- "ubuntu"
    container_id  TEXT,                 -- Docker container ID on remote node
    host_port    INTEGER NOT NULL,      -- port participants connect to
    deployed_at  TEXT NOT NULL,
    stopped_at   TEXT,
    UNIQUE(node_host, host_port)        -- prevent port conflicts
);

-- port_reservations: prevent races when two deploys happen simultaneously
CREATE TABLE port_reservations (
    node_host   TEXT NOT NULL,
    port        INTEGER NOT NULL,
    reserved_at TEXT NOT NULL,
    PRIMARY KEY (node_host, port)
);
```

2.3 Machine Status Lifecycle

```
registered → deploying → active → stopped
                ↓
              error  ←──────────────────┘ (if deploy fails)
```

- `registered` : YAML and assets uploaded; not yet deployed
- `deploying` : Deploy command sent; Docker pull/run in progress on the node
- `active` : Container running; referee is polling it
- `stopped` : Container stopped; referee removes it from poll list

- `error` : Deployment failed; see `deployments.container_id` for Docker logs

---

3. Machine Definition Spec

3.1 koth-machine.yaml

Every machine on the platform, regardless of how it was uploaded, is described by this schema. Organizers who use the web UI form submit data that is normalized into this format before storage.

```
# koth-machine.yaml — complete reference

# ── Identity ──────────────────────────────────────────────────
name: "Vulnerable Nginx"
# Required. Shown in the participant board and admin dashboard.
# Max 64 characters. Must be unique within a competition.

description: "Nginx with a misconfigured sudo rule. Two-stage exploit."
# Optional. Shown to participants. Plain text only; no Markdown.

difficulty: medium
# Required. Allowed values: easy | medium | hard | insane
# The platform does not enforce any difficulty-based scoring
# multipliers by default. This is metadata only.

points_per_tick: 10
# Required. Points awarded each scoring interval while a team
# holds king.txt. Scoring interval is set in referee .env.
# Must be a positive integer.

# ── Ownership Check ───────────────────────────────────────────
king_file: /root/king.txt
# Optional. Defaults to /root/king.txt if omitted.
# The referee SSHes into the container's host node and reads
# this file path. The content (stripped of whitespace) is
# treated as the current owner's team name.
# Change this if your machine's root is not at /root or if
# you want the flag at a custom path.

# ── Container Source — choose exactly ONE ─────────────────────
image: myorg/vuln-nginx:1.0
# Option A: a pre-built image on Docker Hub or a private registry.
# The deployer will run: docker pull myorg/vuln-nginx:1.0
# on the target node before starting the container.
```

```
# dockerfile: ./Dockerfile
# Option B: path to a Dockerfile, relative to this YAML file.
# The deployer will build the image on the platform server,
# tag it, push it to a local registry, then pull on the node.
# See Section 5.3 for build behaviour.

# compose_file: ./docker-compose.yml
# Option C: path to a docker-compose.yml, relative to this YAML.
# The platform normalises the first service's port mapping and
# image reference into the registry. Multi-service composes are
# allowed but only the first service is monitored for king.txt.

# ── Port Mapping ──────────────────────────────────────────────
ports:
  - host: 8080
    container: 80
# Required. At least one port mapping.
# host: the port that participants will connect to.
#   The platform's port manager assigns the actual host port
#   dynamically. This value is used as a HINT — the platform
#   will use it if available, otherwise it picks the next free
#   port in the configured range.
# container: the port the service listens on inside the container.
# Multiple entries are allowed for machines that expose several
# services (e.g., SSH on 22 and a web app on 80).

# ── Metadata ──────────────────────────────────────────────────
tags:
  - web
  - privesc
  - nginx
  - sudo
# Optional. Used for filtering in the admin dashboard.
# No enforced taxonomy; free-form strings.

hints:
  - "Check the web server version carefully."
  - "What binaries can www-data run as root?"
# Optional. Shown to participants after a configurable delay
# or on demand. Plain text only.
```

3.2 Validation Rules

The upload handler runs these checks before accepting a file. Any failure returns HTTP 422 with a specific error message.

Field	Rule
`name`	1–64 characters; no `/:*?"<>
`difficulty`	Must be one of the four allowed values
`points_per_tick`	Integer ≥ 1
`king_file`	Must be an absolute path starting with `/`
Container source	Exactly one of `image`, `dockerfile`, `compose_file` must be set
`ports[].host`	1–65535; must not be 22, 80, 443, 8000, or 9000 (reserved)
`ports[].container`	1–65535
`tags`	List of strings; max 10 items; each max 32 chars
`hints`	List of strings; max 10 items; each max 512 chars

3.3 Normalization from compose_file

When an organizer uploads a `docker-compose.yml`, the platform reads the first service entry and extracts:

```
# platform-server/engine/normalizer.py

import yaml  # always safe_load — see Section 12.1

def normalize_compose(compose_path: str) -> dict:
    with open(compose_path) as f:
        data = yaml.safe_load(f)

    services = data.get("services", {})
    if not services:
        raise ValueError("compose_file has no services defined")

    first_service = next(iter(services.values()))

    image = first_service.get("image")
    ports = []
    for p in first_service.get("ports", []):
        # docker-compose short syntax: "8080:80"
        if isinstance(p, str) and ":" in p:
            host, container = p.split(":", 1)
            ports.append({"host": int(host), "container": int(container)})
```

```
    return {"image": image, "ports": ports}
```

Only `image` and `ports` are extracted from the compose file. All other machine metadata (name, description, difficulty, hints) must be provided either in the web UI form or in a companion `koth-machine.yaml` uploaded alongside the compose file.

---

4. Upload System

4.1 FastAPI File Upload

The upload API is built on FastAPI's `UploadFile` class, which is a subclass of Starlette's `UploadFile`. It stores uploaded files as temporary `SpooledTemporaryFile` objects — files under `spool_max_size` bytes stay in memory; larger files spill to disk automatically.

Reference: fastapi.tiangolo.com/tutorial/request-files

```
# platform-server/api/upload.py

from fastapi import APIRouter, UploadFile, File, Form, HTTPException
from typing import Annotated
import shutil, uuid, os, yaml

router = APIRouter()

UPLOAD_DIR = "platform-server/uploads"
MAX_FILE_SIZE_BYTES = 500 * 1024 * 1024  # 500 MB — adjust for your images

@router.post("/api/machines/upload")
async def upload_machine(
    yaml_file: Annotated[UploadFile, File(description="koth-machine.yaml")],
    asset: Annotated[UploadFile | None, File(description="Dockerfile or docker-co
):
    # 1. Check file size before reading fully into memory
    #    UploadFile.size is set by Starlette from Content-Length if provided.
    #    Do not rely on it alone — validate after reading.
    if yaml_file.size and yaml_file.size > 1024 * 1024:
        raise HTTPException(422, "YAML file must be under 1 MB")

    # 2. Read YAML — always use safe_load (see Section 12.1)
    raw = await yaml_file.read()
    try:
        spec = yaml.safe_load(raw)
```

```
    except yaml.YAMLError as e:
        raise HTTPException(422, f"Invalid YAML: {e}")

    # 3. Validate spec fields
    validate_machine_spec(spec)  # raises HTTPException on failure

    # 4. Save to uploads directory with stable machine ID
    machine_id = str(uuid.uuid4())
    machine_dir = os.path.join(UPLOAD_DIR, machine_id)
    os.makedirs(machine_dir)

    yaml_path = os.path.join(machine_dir, "koth-machine.yaml")
    with open(yaml_path, "wb") as f:
        f.write(raw)

    if asset:
        # Validate filename — do NOT trust asset.filename directly
        safe_name = sanitize_filename(asset.filename)
        asset_path = os.path.join(machine_dir, safe_name)
        with open(asset_path, "wb") as f:
            shutil.copyfileobj(asset.file, f)

    # 5. Write to registry
    register_machine(machine_id, spec, machine_dir)

    return {"machine_id": machine_id, "status": "registered"}
```

Why `shutil.copyfileobj` instead of `await asset.read()`:
`await asset.read()` loads the entire file into memory at once. `shutil.copyfileobj` streams in chunks (default 16 KB). For Docker image tarballs that can be several GB, always use `copyfileobj`.

4.2 Filename Sanitization

Never trust `UploadFile.filename` directly. Browsers may send a path like `../../etc/passwd` or a Windows path like `C:\Users\attacker\payload.sh`. Sanitize before use:

```
import os

def sanitize_filename(filename: str | None) -> str:
    if not filename:
        raise HTTPException(422, "Asset filename is required")
    # os.path.basename strips any directory components
    name = os.path.basename(filename)
```

```
    # Allow only safe characters
    allowed = set("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
    if not all(c in allowed for c in name):
        raise HTTPException(422, f"Filename '{name}' contains invalid characters"
    if name in ("", ".", ".."):
        raise HTTPException(422, "Invalid filename")
    return name
```

4.3 Upload Methods Compared

Method	How it works	Best for
YAML + pre-built image	Upload YAML. Image is pulled from registry on deploy.	Public Docker Hub images; organizers who already have images published
YAML + Dockerfile	Upload YAML and Dockerfile (optionally with a build context zip). Platform builds on the platform server, pushes to local registry.	Organizers building custom machines locally
docker-compose.yml	Upload compose file. Platform extracts image and port config.	Organizers migrating from existing compose-based setups
Web UI form	Fill in fields; platform generates YAML internally. No file needed.	Organizers who are not comfortable with YAML

4.4 Web UI Form → YAML

The web UI form posts to `POST /api/machines/create` with `application/json` body. The handler generates a `koth-machine.yaml` file internally and writes it to `uploads/{machine_id}/`. This keeps the registry format consistent regardless of how the machine was registered.

---

5. Deployment Engine

5.1 Docker SDK for Python

The deployer uses the official Docker SDK for Python (`docker` package, currently 7.1.0). The SDK wraps the Docker Engine API and communicates over the local Unix socket (`/var/run/docker.sock`) or a TCP endpoint for remote daemons.

Install: `pip install docker`
Reference: docker-py.readthedocs.io

Connecting to a local daemon:

```
import docker

# Reads DOCKER_HOST env var; falls back to /var/run/docker.sock
client = docker.from_env()
```

Connecting to a remote daemon (TCP with TLS):

```
import docker

client = docker.DockerClient(
    base_url="tcp://192.168.1.10:2376",
    tls=True,
    # Requires TLS certs on the remote daemon.
    # See: https://docs.docker.com/engine/security/protect-access/
)
```

> **Warning:** Never expose Docker's TCP port without TLS. Port 2375 (plain TCP) gives root-equivalent access to the host. Always use 2376 with mutual TLS, or prefer the SSH-based approach below (Section 5.2).

5.2 Remote Deployment via SSH + Docker

Rather than exposing Docker over TCP, the platform deploys to remote nodes by SSHing in and running Docker commands over the SSH channel. This avoids needing to configure TLS certificates on every node and keeps the Docker daemon bound to its local socket only.

The deployer uses **Paramiko** for SSH.

Install: `pip install paramiko`
Reference: paramiko.org/api/client.html

```
# platform-server/engine/deployer.py

import paramiko
import json

def deploy_to_node(
    node_host: str,
    ssh_user: str,
    ssh_key_path: str,
```

```
    image: str,
    container_name: str,
    port_mapping: dict,  # {host_port: container_port}
    king_file: str,
) -> str:
    """
    Deploy a container on a remote node via SSH.
    Returns the Docker container ID on success.
    Raises RuntimeError on failure.
    """
    ssh = paramiko.SSHClient()
    # AutoAddPolicy accepts any host key.
    # For production, use RejectPolicy and maintain a known_hosts file.
    # See Section 12.2 for the security tradeoff.
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    try:
        ssh.connect(
            hostname=node_host,
            username=ssh_user,
            key_filename=ssh_key_path,
            timeout=10,          # connection timeout in seconds
            banner_timeout=10,   # time to wait for SSH banner
        )

        # Build port flags: "-p 10085:80"
        port_flags = " ".join(
            f"-p {hp}:{cp}" for hp, cp in port_mapping.items()
        )

        # Pull the image first, separate from run
        pull_cmd = f"docker pull {image}"
        _exec(ssh, pull_cmd, timeout=120)  # image pulls can be slow

        # Run the container
        run_cmd = (
            f"docker run -d "
            f"--name {container_name} "
            f"--restart unless-stopped "
            f"{port_flags} "
            f"{image}"
        )
        stdout = _exec(ssh, run_cmd, timeout=30)
        container_id = stdout.strip()

        return container_id
```

```
    finally:
        ssh.close()


def _exec(ssh: paramiko.SSHClient, command: str, timeout: int) -> str:
    """
    Execute a command over SSH and return stdout.
    Raises RuntimeError if the command exits non-zero.

    IMPORTANT: Read stderr before or concurrently with stdout to avoid
    the deadlock described in paramiko issue #1778.
    See Section 11.2 for full explanation.
    """
    stdin, stdout, stderr = ssh.exec_command(command, timeout=timeout)
    stdin.close()  # we never write to stdin

    # Read stderr first (or at least drain it) to prevent the
    # transport-layer deadlock when both buffers fill up.
    err = stderr.read().decode("utf-8", errors="replace")
    out = stdout.read().decode("utf-8", errors="replace")

    exit_code = stdout.channel.recv_exit_status()
    if exit_code != 0:
        raise RuntimeError(
            f"Command failed (exit {exit_code}).\n"
            f"Command: {command}\n"
            f"Stderr: {err}\n"
            f"Stdout: {out}"
        )
    return out
```

5.3 Building from a Dockerfile

When an organizer uploads a Dockerfile, the platform builds the image on the platform server, not on the deployment node. This keeps node setup minimal (they only need Docker installed; no build tools required).

```
# platform-server/engine/builder.py

import docker, os

def build_and_push(
    dockerfile_dir: str,
    tag: str,
    registry_host: str,  # e.g. "localhost:5000" for a local registry
) -> str:
```

```
    """
    Build a Docker image from a Dockerfile directory.
    Push to a local registry so nodes can pull it.
    Returns the full image reference: registry_host/tag
    """
    client = docker.from_env()

    full_tag = f"{registry_host}/{tag}"

    # client.images.build returns (Image, generator_of_log_lines)
    # The generator must be consumed to avoid the build hanging.
    image, logs = client.images.build(
        path=dockerfile_dir,
        tag=full_tag,
        rm=True,           # remove intermediate containers after build
        forcerm=True,      # remove even on failure
    )

    # Consume and surface build logs
    for chunk in logs:
        if "stream" in chunk:
            print(chunk["stream"], end="")
        if "error" in chunk:
            raise RuntimeError(f"Build failed: {chunk['error']}")

    # Push to local registry
    # Requires a local registry running at registry_host.
    # Start one with: docker run -d -p 5000:5000 registry:2
    push_output = client.images.push(full_tag)
    print(push_output)

    return full_tag
```

Running a local Docker registry:

```
docker run -d \
  --name koth-registry \
  --restart unless-stopped \
  -p 5000:5000 \
  -v /srv/registry:/var/lib/registry \
  registry:2
```

Nodes must be configured to trust this registry as insecure (since it runs without TLS on a LAN). Add to `/etc/docker/daemon.json` on each node:

```
{
  "insecure-registries": ["192.168.1.100:5000"]
}
```

Then restart Docker: `sudo systemctl restart docker`

5.4 Undeploying a Machine

```
def undeploy_from_node(
    node_host: str,
    ssh_user: str,
    ssh_key_path: str,
    container_name: str,
):
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(node_host, username=ssh_user, key_filename=ssh_key_path)
    try:
        # Stop then remove. Separate commands so stop errors don't
        # prevent removal of an already-exited container.
        try:
            _exec(ssh, f"docker stop {container_name}", timeout=30)
        except RuntimeError:
            pass  # container may already be stopped
        _exec(ssh, f"docker rm {container_name}", timeout=10)
    finally:
        ssh.close()
```

---

6. Port Manager

6.1 Problem

When a machine is deployed, the platform must assign it a host port on the target node. Two concurrent deployments to the same node must not receive the same port. The assignment must survive a platform server restart.

6.2 Design

The port manager uses SQLite's `UNIQUE` constraint on `(node_host, port)` as an atomic reservation mechanism. There is no in-memory lock — the database is the lock. This works correctly even with multiple concurrent API requests because SQLite serializes writes.

```
# platform-server/engine/port_manager.py

import sqlite3, datetime

PORT_RANGE_START = 10000
PORT_RANGE_END   = 20000

def reserve_port(db_path: str, node_host: str, preferred_port: int | None = None)
    """
    Reserve the next available port on node_host.
    If preferred_port is given and free, reserves that port.
    Returns the reserved port number.
    Raises RuntimeError if no ports are available.
    """
    with sqlite3.connect(db_path) as conn:
        conn.execute("PRAGMA journal_mode=WAL")
        conn.execute("PRAGMA busy_timeout=5000")

        # Fetch all currently reserved ports for this node
        taken = {
            row[0] for row in conn.execute(
                "SELECT port FROM port_reservations WHERE node_host = ?",
                (node_host,)
            )
        }
        # Also check the deployments table for active containers
        active = {
            row[0] for row in conn.execute(
                "SELECT host_port FROM deployments WHERE node_host = ? AND stoppe
                (node_host,)
            )
        }
        reserved = taken | active

        # Try preferred port first
        candidates = []
        if preferred_port and preferred_port not in reserved:
            candidates.append(preferred_port)
        candidates += [p for p in range(PORT_RANGE_START, PORT_RANGE_END) if p no

        if not candidates:
            raise RuntimeError(f"No free ports on {node_host}")

        chosen = candidates[0]
        conn.execute(
            "INSERT INTO port_reservations (node_host, port, reserved_at) VALUES
            (node_host, chosen, datetime.datetime.utcnow().isoformat())
```

```
        )
        conn.commit()
        return chosen


def release_port(db_path: str, node_host: str, port: int):
    """Remove a port reservation when a container is stopped."""
    with sqlite3.connect(db_path) as conn:
        conn.execute(
            "DELETE FROM port_reservations WHERE node_host = ? AND port = ?",
            (node_host, port)
        )
        conn.commit()
```

6.3 Reserved Ports

These ports are never assigned by the port manager regardless of range configuration:

Port	Reason
22	SSH — node management
8000	Platform-server admin dashboard
9000	Participant board
5000	Local Docker registry

---

7. Adaptive Referee

7.1 What Changed from v1

In v1, the referee loaded SSH targets from `NODE_SSH_TARGETS` in `.env` at startup. Those targets were static for the entire competition.

In v2, the referee calls `GET /api/machines/active` at the start of each scoring interval. This returns the current list of deployed, active machines. The referee diffs this against its current poll list and adds or removes machines without restarting.

7.2 Hot-Reload Logic

```
# referee-server/poller.py
```

```
import asyncio, paramiko, httpx, datetime, logging

logger = logging.getLogger(__name__)

class Poller:
    def __init__(self, platform_url: str, ssh_key_path: str, interval_seconds: in
        self.platform_url = platform_url
        self.ssh_key_path = ssh_key_path
        self.interval = interval_seconds
        self._active_machines: dict[str, dict] = {}  # machine_id → machine info

    async def run_forever(self):
        while True:
            try:
                await self._reload_machines()
                await self._poll_all()
            except Exception as e:
                logger.error(f"Poller error: {e}", exc_info=True)
            await asyncio.sleep(self.interval)

    async def _reload_machines(self):
        """Fetch the current active machine list from the registry."""
        async with httpx.AsyncClient() as client:
            resp = await client.get(
                f"{self.platform_url}/api/machines/active",
                timeout=5.0
            )
            resp.raise_for_status()
            machines = {m["id"]: m for m in resp.json()}

        added   = set(machines) - set(self._active_machines)
        removed = set(self._active_machines) - set(machines)

        for mid in added:
            logger.info(f"Referee: tracking new machine {machines[mid]['name']} "
                        f"at {machines[mid]['node_host']}:{machines[mid]['host_po
        for mid in removed:
            logger.info(f"Referee: dropping machine {self._active_machines[mid]['

        self._active_machines = machines

    async def _poll_all(self):
        """Read king.txt on every active machine and submit scores."""
        tasks = [self._poll_one(m) for m in self._active_machines.values()]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        for machine, result in zip(self._active_machines.values(), results):
```

```
        if isinstance(result, Exception):
            logger.warning(f"Poll failed for {machine['name']}: {result}")
        else:
            owner, timestamp = result
            if owner:
                await self._record_score(machine["id"], owner, timestamp)

async def _poll_one(self, machine: dict) -> tuple[str | None, str]:
    """
    SSH to the machine's host node; read king_file.
    Returns (owner_team_name, timestamp) or (None, timestamp) if unowned.

    Runs in a thread pool because Paramiko is synchronous.
    """
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, self._ssh_read_king_file, machine

def _ssh_read_king_file(self, machine: dict) -> tuple[str | None, str]:
    """Synchronous SSH read. Called from executor thread."""
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    timestamp = datetime.datetime.utcnow().isoformat()

    try:
        ssh.connect(
            hostname=machine["node_host"],
            username=machine["node_ssh_user"],
            key_filename=self.ssh_key_path,
            timeout=5,
            banner_timeout=5,
        )

        # Use docker exec to read the file from inside the container.
        # This avoids needing SSH inside the container itself.
        cmd = (
            f"docker exec {machine['container_id']} "
            f"cat {machine['king_file']} 2>/dev/null || true"
        )
        _, stdout, stderr = ssh.exec_command(cmd, timeout=5)
        stderr.read()  # drain stderr to prevent deadlock (paramiko #1778)
        content = stdout.read().decode("utf-8", errors="replace").strip()

        owner = content if content else None
        return (owner, timestamp)

    except (paramiko.SSHException, OSError) as e:
        raise RuntimeError(f"SSH error for {machine['name']}: {e}") from e
```

```
        finally:
            ssh.close()
```

7.3 GET /api/machines/active Response Shape

```
[
  {
    "id": "a3f2c1d0-...",
    "name": "Vulnerable Nginx",
    "node_host": "192.168.1.10",
    "node_ssh_user": "ubuntu",
    "container_id": "b3e1f2a4c5d6...",
    "host_port": 10085,
    "king_file": "/root/king.txt",
    "points_per_tick": 10
  }
]
```

---

8. Database Layer

8.1 SQLite + WAL Mode

The platform uses SQLite as its database. SQLite is appropriate here because:

- The platform server is a single process running on one host
- Write volume is low (one write per deployment, one write per scoring tick per machine)
- No distributed state is needed
- No separate database server to manage

WAL mode (Write-Ahead Logging) is enabled on every connection. Under the default DELETE journal mode, any writer blocks all readers. WAL allows multiple concurrent readers while a single writer appends to a separate WAL file.

Reference: sqlite.org/wal.html

```
# platform-server/engine/db.py

import sqlite3

def get_connection(db_path: str) -> sqlite3.Connection:
```

```
    conn = sqlite3.connect(db_path, check_same_thread=False)
    conn.row_factory = sqlite3.Row  # access columns by name

    # Enable WAL mode. This is persistent — only needs to be set once
    # per database file, but setting it on each new connection is harmless.
    conn.execute("PRAGMA journal_mode=WAL")

    # Without busy_timeout, a concurrent write raises
    # sqlite3.OperationalError: database is locked immediately.
    # 5000 ms is enough for the other writer to finish under normal load.
    conn.execute("PRAGMA busy_timeout=5000")

    return conn
```

WAL limitations to be aware of:

- WAL requires that all processes accessing the database are on the same host machine. It will not work correctly over NFS or other network filesystems. If you mount the database file over the network, use DELETE journal mode instead.
- WAL creates two additional files alongside the database: `platform.db-wal` and `platform.db-shm`. These must be kept with the `.db` file. Deleting them while the database is in use causes corruption.
- WAL files grow until a checkpoint writes them back to the main database file. A passive checkpoint happens automatically, but you can run `PRAGMA wal_checkpoint(TRUNCATE)` during a maintenance window to reclaim disk space.

8.2 Migrations

Use plain SQL migration files numbered sequentially. Run them in order at startup:

```
platform-server/migrations/
  001_initial_schema.sql
  002_add_hints_column.sql
  003_add_port_reservations.sql
```

```
# platform-server/engine/db.py (continued)

import os, glob

def apply_migrations(db_path: str, migrations_dir: str):
    conn = get_connection(db_path)
    conn.execute(
        "CREATE TABLE IF NOT EXISTS schema_migrations "
```

```
        "(version TEXT PRIMARY KEY, applied_at TEXT)"
    )
    conn.commit()

    applied = {row[0] for row in conn.execute("SELECT version FROM schema_migrati

    files = sorted(glob.glob(os.path.join(migrations_dir, "*.sql")))
    for path in files:
        version = os.path.basename(path)
        if version not in applied:
            with open(path) as f:
                sql = f.read()
            conn.executescript(sql)
            conn.execute(
                "INSERT INTO schema_migrations (version, applied_at) VALUES (?, d
                (version,)
            )
            conn.commit()
            print(f"Applied migration: {version}")
```

---

9. Web UI

9.1 Pages

Page	URL	Purpose
Machine Upload	`/upload`	Upload YAML, Dockerfile, or compose file; or fill out the web form
Machine Registry	`/machines`	Table of all registered machines with status, difficulty, and actions
Deploy	`/deploy`	Select a machine and a target node; click Deploy
Node Health	`/nodes`	Live status of each node: running containers, free ports, disk/CPU
Admin Dashboard	`/admin`	Scoreboard management; team controls; competition lifecycle
Participant Board	`:9000/`	Public-facing leaderboard, rules, organizer notices

9.2 Web Form → YAML Mapping

The upload form on `/upload` generates a `koth-machine.yaml` equivalent when submitted. Fields map directly to the YAML spec:

Form Field	YAML Key	Type	Notes
Machine Name	`name`	text	
Description	`description`	textarea	
Difficulty	`difficulty`	select	
Points/Tick	`points_per_tick`	number	
King File Path	`king_file`	text	Defaults to `/root/king.txt`
Docker Image	`image`	text	Leave blank if uploading Dockerfile
Hints	`hints`	textarea	One hint per line
Tags	`tags`	text	Comma-separated
Host Port	`ports[0].host`	number	
Container Port	`ports[0].container`	number	
Upload file	`dockerfile` or `compose_file`	file	Only if not using Docker image

---

10. API Reference

All endpoints on `platform-server` require the `X-Admin-Key` header unless marked `[public]`.

10.1 Machine Registry

```
POST   /api/machines/upload
       Multipart form: yaml_file (required), asset (optional)
```

```
        Response: { "machine_id": "...", "status": "registered" }

POST    /api/machines/create
        JSON body: full machine spec
        Response: { "machine_id": "...", "status": "registered" }

GET     /api/machines
        Query params: status, difficulty, tag
        Response: [ { machine object } ]

GET     /api/machines/{id}
        Response: { machine object with full spec }

DELETE /api/machines/{id}
        Only allowed if status is "registered" or "stopped"
        Response: 204 No Content

GET     /api/machines/active   [public — referee uses this]
        Response: [ { active machine with deployment info } ]
```

10.2 Deployment

```
POST    /api/deploy
        Body: { "machine_id": "...", "node_host": "...", "preferred_port": 10085 }
        Response: { "deployment_id": "...", "host_port": 10085, "status": "deployi
        Note: Deploy is asynchronous. Poll GET /api/deploy/{id} for status.

GET     /api/deploy/{deployment_id}
        Response: { "status": "active"|"deploying"|"error", "container_id": "...",

DELETE /api/deploy/{deployment_id}
        Stops and removes the container on the node.
        Releases the port reservation.
        Updates machine status to "stopped".
```

10.3 Nodes

```
GET     /api/nodes
        Response: list of configured nodes with SSH connectivity status

GET     /api/nodes/{host}/health
        SSHes to node, returns: running containers, disk usage, free ports
```

10.4 Competition (unchanged from v1)

```
POST   /api/competition/start
POST   /api/competition/stop
POST   /api/competition/pause
GET    /api/runtime
GET    /api/recover/validate
POST   /api/recover/redeploy
```

---

11. Known Issues and Pitfalls

This section documents real problems encountered in development and production, with references to upstream issue trackers where applicable.

11.1 Docker SDK: Port Mapping Argument Order

The Docker SDK for Python uses an inverted port notation compared to the CLI. In the CLI you write `docker run -p HOST:CONTAINER`. In the SDK, the dictionary key is the **container port** and the value is the **host port**:

```
# CLI:  docker run -p 10085:80
# SDK:
client.containers.run(
    image,
    ports={"80/tcp": 10085}  # container_port/protocol: host_port
)
```

This is explicitly noted in the SDK documentation. Getting it backwards results in the container binding to port 80 on the host (or failing if port 80 is taken) rather than port 10085.

Reference: docker-py.readthedocs.io/en/stable/containers.html

11.2 Paramiko: stdout.read() Deadlock

`ssh.exec_command()` returns `(stdin, stdout, stderr)` channel file objects. If you call `stdout.read()` before reading `stderr`, and the remote command produces output on both channels simultaneously, you can hit a transport-layer deadlock:

- The remote process fills the stderr buffer (window size ≈ 2 MiB)
- The SSH transport thread blocks trying to write more stderr

- Your application thread blocks reading stdout
- Neither side makes progress

The fix: Always read `stderr` before or instead of letting it buffer up. The safest pattern is to read `stderr` first, then `stdout`:

```
# WRONG — can deadlock if command writes to both channels
out = stdout.read()  # blocks if stderr buffer is full
err = stderr.read()

# CORRECT — drain stderr first
err = stderr.read()
out = stdout.read()
```

Alternatively, pass `get_pty=True` to `exec_command` to merge stdout and stderr into a single channel (PTY), which eliminates the two-buffer deadlock but changes output formatting.

Reference: github.com/paramiko/paramiko/issues/1778

11.3 Paramiko: exec_command Does Not Return Until EOF

`stdout.read()` blocks until the remote command closes its stdout. Commands that hold stdout open (interactive shells, daemons, `tail -f`) will block indefinitely. Always set a timeout on `exec_command`:

```
stdin, stdout, stderr = ssh.exec_command(command, timeout=30)
```

The `timeout` is applied to channel read operations. If the command does not close stdout within `timeout` seconds of inactivity, `paramiko.buffered_pipe.PipeTimeout` is raised.

This is different from the connection timeout set in `ssh.connect(timeout=...)`. The connection timeout applies to establishing the TCP connection. The channel timeout applies to waiting for data on the channel.

11.4 SQLite: WAL Files Must Not Be Separated from the Database

If you copy the database file without copying `platform.db-wal` and `platform.db-shm`, uncommitted transactions are lost and the database may be corrupt. Use the SQLite backup API (via `.backup()`) or the `VACUUM INTO` command to create a consistent backup:

```
# Safe backup using the SQLite CLI
sqlite3 platform.db "VACUUM INTO '/backup/platform_backup.db'"
```

Reference: sqlite.org/wal.html — "Avoiding Resource Starvation"

11.5 SQLite: WAL Does Not Work on Network Filesystems

WAL mode requires shared memory primitives (`platform.db-shm`) that do not work reliably over NFS, CIFS, or other network filesystems. If the database file is on a network-mounted volume, WAL mode will silently fall back to rollback journal mode or cause corruption. Keep the SQLite database on a local filesystem.

Reference: sqlite.org/wal.html — "Limitations"

11.6 Docker Build Context Size

When building from a Dockerfile with `client.images.build(path=...)` , the Docker SDK sends the entire contents of `path` to the Docker daemon as a build context tarball. If the directory contains large files (e.g., database dumps, VM images), this will be slow and may exhaust memory on the platform server.

Always include a `.dockerignore` file at the root of the build context directory listing files to exclude. The syntax is identical to `.gitignore` :

```
# .dockerignore
*.tar
*.iso
*.vmdk
node_modules/
__pycache__/
```

11.7 Container Names Must Be Unique Per Node

`docker run --name` fails if a container with that name already exists on the node, even if it is stopped. The deployer generates container names as `koth-{machine_id[:8]}` and calls `docker rm` on undeploy. If a deploy fails midway (e.g., the image pulled but the `run` command failed), the partial container may be left behind with the name reserved. The recovery flow must call `docker rm -f {container_name}` before retrying.

```
# deployer.py: before every deploy attempt, clean up any stale container
def _cleanup_stale(ssh, container_name: str):
    try:
        _exec(ssh, f"docker rm -f {container_name}", timeout=10)
    except RuntimeError:
        pass  # container did not exist; that's fine
```

11.8 Port Manager Race Condition Window

There is a small window between `reserve_port()` checking for a free port and inserting the reservation where two concurrent requests could both see the same port as free. The `UNIQUE` constraint on `(node_host, port)` in `port_reservations` prevents both from succeeding — the second will get `sqlite3.IntegrityError`. The caller should catch this and retry with a new port:

```
import sqlite3

def reserve_port_with_retry(db_path, node_host, preferred_port=None, max_attempts
    for attempt in range(max_attempts):
        try:
            return reserve_port(db_path, node_host, preferred_port)
        except sqlite3.IntegrityError:
            preferred_port = None  # clear preference; pick any free port
            continue
    raise RuntimeError(f"Could not reserve a port after {max_attempts} attempts")
```

---

12. Security Considerations

12.1 PyYAML: Never Use yaml.load() on Untrusted Input

PyYAML's `yaml.load()` function can execute arbitrary Python code if given a malicious YAML file. This is not a bug — it is documented behaviour. The `!!python/object/apply:os.system` YAML tag, among others, allows code execution. This affects `yaml.load()` with the default `FullLoader` in versions before 5.4 and with `Loader=yaml.Loader` in all versions.

Always use `yaml.safe_load()` for any organizer-uploaded content. `safe_load()` only constructs standard Python objects (dicts, lists, strings, numbers, booleans, `None`) and rejects any tag-based object construction.

```
# NEVER do this with organizer-uploaded files
config = yaml.load(user_data)                              # dangerous
config = yaml.load(user_data, Loader=yaml.Loader)          # dangerous
config = yaml.load(user_data, Loader=yaml.FullLoader)      # dangerous (pre-5.4)

# Always do this
config = yaml.safe_load(user_data)                         # safe in all versions
```

CVE references: CVE-2017-18342, CVE-2020-1747, CVE-2020-14343
Reference: github.com/yaml/pyyaml/wiki/PyYAML-yaml.load(input)-Deprecation

12.2 Paramiko: Host Key Verification

Using `AutoAddPolicy` (accept any host key on first connection) is vulnerable to a man-in-the-middle attack: an attacker on the same network can intercept the SSH connection on the first deploy and present a forged key. The platform will accept it, and subsequent connections to the same host will trust that forged key.

For a competition network where you control all nodes, the acceptable mitigation is to pre-populate `~/.ssh/known_hosts` on the platform server with each node's host key before running the platform, and use `RejectPolicy` instead:

```
ssh.set_missing_host_key_policy(paramiko.RejectPolicy())

# Pre-populate known_hosts before running the platform:
# ssh-keyscan -H 192.168.1.10 >> ~/.ssh/known_hosts
# ssh-keyscan -H 192.168.1.11 >> ~/.ssh/known_hosts
# ssh-keyscan -H 192.168.1.12 >> ~/.ssh/known_hosts
```

12.3 Container Isolation

The machines on the platform are intentionally vulnerable. Participants will exploit them. Do not run the platform on nodes that also run production services. Each node should be a dedicated VM used exclusively for the competition.

Network-level isolation recommendations:

- Participants can reach `node_host:host_port` for each deployed machine
- Participants must not be able to reach port 22 on any node (this is how the referee manages machines)
- Participants must not be able to reach the platform server admin dashboard on port 8000
- The firewall on each node should block all inter-container traffic unless the machine requires it

12.4 Admin API Key

The `ADMIN_API_KEY` must be set in the platform server's `.env` before starting. All admin API endpoints check for this key in the `X-Admin-Key` header. The platform refuses to start without it unless `ALLOW_UNSAFE_NO_ADMIN_API_KEY=true` is set (for local development only).

Generate a key with sufficient entropy:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

12.5 Docker Image Provenance

When an organizer provides `image: someuser/somerepo:latest` in their YAML, the deployer will pull that image from Docker Hub onto your competition nodes. Malicious images could exfiltrate network data, attack other containers, or harm the host. For public competitions:

- Require organizers to submit images in advance for review
- Pull and scan images on the platform server before allowing deploy
- Use a content-addressable reference (`image: myorg/vuln-nginx@sha256:a3f2...`) instead of a mutable tag to prevent the image from being replaced after review

---

13. Running in Production

13.1 Node Setup

Each competition node needs only:

```
# Ubuntu 22.04+

# Docker Engine (not Docker Desktop)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# SSH server (for platform-server and referee-server access)
sudo systemctl enable --now ssh

# Open required ports in firewall
# 22: SSH (restricted to platform-server and referee-server IP only)
# 10000-20000: participant challenge ports
sudo ufw allow from <platform_server_ip> to any port 22
sudo ufw allow 10000:20000/tcp
sudo ufw enable
```

13.2 Platform Server Setup

```
cd platform-server
```

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Create .env
cat > .env << 'EOF'
ADMIN_API_KEY=<your_key_from_section_12.4>
DB_PATH=./platform.db
UPLOAD_DIR=./uploads
SSH_KEY_PATH=/home/ubuntu/.ssh/koth_deploy_key
PORT_RANGE_START=10000
PORT_RANGE_END=20000
EOF

# Apply database migrations
python manage.py migrate

# Start
uvicorn app:app --host 0.0.0.0 --port 8000
```

13.3 Referee Server Setup

```
cd referee-server
pip install -r requirements.txt

cat > .env << 'EOF'
ADMIN_API_KEY=<same_key_as_platform_server>
PLATFORM_URL=http://localhost:8000
SSH_KEY_PATH=/home/ubuntu/.ssh/koth_deploy_key
SCORE_INTERVAL=30
EOF

uvicorn app:app --host 0.0.0.0 --port 8001
```

13.4 Process Management

Use `systemd` to keep both servers running and restart them on failure.

```
# /etc/systemd/system/koth-platform.service
[Unit]
Description=KoTH Platform Server
After=network.target

[Service]
User=ubuntu
```

```
WorkingDirectory=/opt/koth/platform-server
EnvironmentFile=/opt/koth/platform-server/.env
ExecStart=/opt/koth/platform-server/venv/bin/uvicorn app:app --host 0.0.0.0 --por
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target


sudo systemctl enable --now koth-platform
sudo systemctl enable --now koth-referee
```

13.5 Pre-Competition Checklist

- SSH key deployed to all nodes; `known_hosts` populated on platform server
- `ADMIN_API_KEY` set in both `.env` files
- Local Docker registry running (if using Dockerfile uploads)
- All nodes added to firewall allowlist for SSH from platform server
- At least one machine registered and successfully deployed as a smoke test
- Referee returns healthy status from `GET /api/recover/validate`
- Participant board accessible from the participant network at `:9000`
- Admin dashboard not reachable from participant network

---

14. Contributing a Machine

14.1 Directory Structure

Contributed machines live in `contributed-machines/` grouped by difficulty:

```
contributed-machines/
  easy/
    redis-unauthenticated/
      koth-machine.yaml
      Dockerfile
      setup.sh          (optional: provisions vulns inside container)
      README.md         (walkthrough for organizers, not participants)
  medium/
```

```
    ...
  hard/
    ...
  insane/
    ...
```

14.2 Requirements for Contributed Machines

1. **The machine must be self-contained.** Everything needed to build and run it must be in the directory. No references to external servers or private registries.
2. **`/root/king.txt` must exist and be world-writable after privilege escalation.** The file is created at container startup with content `unclaimed`. A participant who gets root can overwrite it.
3. **The machine must have a two-stage exploit path:** initial foothold → privilege escalation to root. Single-step root is only acceptable for `easy` difficulty.
4. **The README.md must include:** initial vector, privesc vector, CVE references if applicable, and a note on which services run on which container ports.
5. **No outbound internet calls during runtime.** The container must not depend on external package downloads or API calls once running.

14.3 Minimal Dockerfile for a Contributed Machine

```
FROM ubuntu:22.04

# Disable apt prompts during build
ENV DEBIAN_FRONTEND=noninteractive

# Install services needed for the vulnerability
RUN apt-get update && apt-get install -y \
    openssh-server \
    nginx \
    sudo \
    && rm -rf /var/lib/apt/lists/*

# Create the king.txt file at a known location
RUN mkdir -p /root && echo "unclaimed" > /root/king.txt && chmod 666 /root/king.t

# Configure the intentional vulnerability
RUN echo "www-data ALL=(root) NOPASSWD: /usr/bin/find" >> /etc/sudoers

# Start SSH and Nginx
```

```
COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

```
#!/bin/bash
# entrypoint.sh
service ssh start
service nginx start
tail -f /dev/null  # keep container alive
```

14.4 koth-machine.yaml for a Contributed Machine

```yaml
name: "Redis No-Auth"
description: "Redis 6 running with no password. Two escalation paths available."
difficulty: easy
points_per_tick: 10
king_file: /root/king.txt
image: ""  # leave blank; Dockerfile is in this directory
dockerfile: ./Dockerfile
ports:
  - host: 6379
    container: 6379
tags: [redis, database, nosql, rce]
hints:
  - "Redis allows saving its database file to arbitrary paths."
  - "What paths does the redis user have write access to?"
```

Documentation maintained by the KoTH Orchestrator project.
Upstream references: Docker SDK · Paramiko · FastAPI · SQLite WAL · PyYAML safety